Module 5: Code Optimization — C Programming Question Bank

Conventions used in all questions

- Instructions are numbered from 1 (three-address code) unless stated otherwise.
- All output sets are printed in ascending order. An empty set is printed as `{}`.
- Input is always valid. Use `fgets` / `sscanf` to read lines.

PART A: BASIC BLOCKS AND FLOW GRAPHS

Q1. Basic Block Partitioning (Leader Algorithm)

Idea: A *leader* is (a) the first instruction, (b) any instruction that is the target of a jump, (c) any instruction immediately after a jump. A basic block runs from one leader up to the instruction before the next leader.

Problem Statement Write a C program that reads a three-address code program and partitions it into basic blocks.

The program should:

1. Read `n`, then `n` instructions in the form `k: instruction`.
2. Instruction types: `x = y op z`, `x = y`, `goto k`, `if x relop y goto k`.
3. Find all leaders using the three leader rules.
4. Print the leaders in ascending order.
5. Print each basic block as `Bi: start-end`.

Input Constraints

- $1 \leqslant n \leqslant 50$
- Jump targets are valid instruction numbers.

Public Test 1

```
input=
8
1: i = 1
2: j = 1
3: t1 = 10 * i
4: t2 = t1 + j
5: j = j + 1
6: if j <= 10 goto 3
7: i = i + 1
8: if i <= 10 goto 2

output=
Leaders: 1 2 3 7
B1: 1-1
B2: 2-2
B3: 3-6
B4: 7-8
```

Public Test 2

```
input=
5
1: a = 1
2: if a > 5 goto 5
3: a = a + 1
4: goto 2
5: b = a

output=
Leaders: 1 2 3 5
B1: 1-1
B2: 2-2
B3: 3-4
B4: 5-5
```

Q2. Flow Graph Construction

Idea: There is an edge from block B to block C if (a) the last instruction of B jumps (conditionally or unconditionally) to the first instruction of C, or (b) C immediately follows B and B does not end with an unconditional `goto`.

Problem Statement Extend Q1. After partitioning into basic blocks, print all edges of the flow graph.

The program should:

1. Read the same input format as Q1.

2. Print edges as `Bi->Bj`, sorted by source block then destination block.
3. A block ending in an unconditional `goto` has no fall-through edge.
4. A block ending in a conditional jump has both the jump edge and the fall-through edge (if a next block exists).

Public Test 1 (input same as Q1 Test 1)

```
output=
B1->B2
B2->B3
B3->B3
B3->B4
B4->B2
```

Public Test 2 (input same as Q1 Test 2)

```
output=
B1->B2
B2->B3
B2->B4
B3->B2
```

PART B: OPTIMIZATION OF BASIC BLOCKS

Q3. DAG Construction and Common Subexpression Elimination

Idea: Build a DAG for the block. If a node with the same operator and the same two children already exists, reuse it (that is common subexpression elimination). Each node holds a list of variables (labels). Generate code from the DAG.

Problem Statement Write a C program that builds a DAG for a basic block and prints the optimized three-address code.

The program should:

1. Read `n`, then `n` statements of the form `x = y op z` (op is `+ - * /`).
2. For each statement, find the current node of `y` and `z` (create a leaf node if the variable has not been seen).
3. If a node with the same operator and same children exists, attach `x` to that node. Otherwise create a new node labelled `x`.
4. Process nodes in the order they were created.
5. For each node, the **first** label gets the computation. Every other label gets a copy `label = first_label`, printed right after.
6. An operand is printed using the first label of its node (or the variable name if it is a leaf).

7. Assume all variables are live at the end of the block.

Input Constraints

- $1 \leqslant n \leqslant 30$
- Variables are single lowercase letters or short names such as `t1`.

Public Test 1

```
input=
4
a = b + c
b = a - d
c = b + c
d = a - d

output=
a = b + c
b = a - d
d = b
c = b + c
```

Public Test 2

```
input=
3
t1 = a + b
t2 = a + b
t3 = t1 * t2

output=
t1 = a + b
t2 = t1
t3 = t1 * t1
```

Public Test 3 (no false elimination, because `b` and `c` are redefined)

```
input=
4
a = b + c
b = b - d
c = c + d
e = b + c

output=
a = b + c
b = b - d
c = c + d
e = b + c
```

Q4. Peephole Optimizer

Idea: Slide a small window over the target code and replace inefficient patterns with better ones. Repeat until nothing changes.

Problem Statement Write a C program that applies the following peephole rules repeatedly until no rule can be applied.

Instruction set: `LD R,x`, `ST x,R`, `ADD R,v`, `SUB R,v`, `MUL R,v`, `DIV R,v`, `SHL R,k`, `JMP L`, and labels written as `L:`.

Rules:

1. **Redundant load:** `ST x, R` immediately followed by `LD R, x` → delete the `LD`.
2. **Algebraic simplification:** `ADD R, 0`, `SUB R, 0`, `MUL R, 1`, `DIV R, 1` → delete the instruction.
3. **Strength reduction:** `MUL R, 2^k` for 2, 4, 8 → `SHL R, k`.
4. **Jump to next instruction:** `JMP L` immediately followed by `L:` → delete the `JMP`.
5. **Unreachable code:** instructions after a `JMP` up to the next label are deleted.

The program should read `n` and then `n` instructions, and print the optimized code.

Input Constraints

- $1 \leqslant n \leqslant 50$
- Registers are named `R1`, `R2`, ...

Public Test 1

```
input=
7
LD R1, a
ADD R1, 0
ST x, R1
LD R1, x
MUL R1, 4
JMP L1
L1:

output=
LD R1, a
ST x, R1
SHL R1, 2
L1:
```

Public Test 2 (rules must be applied in more than one pass)

```
input=
5
JMP L2
ADD R1, R2
ST y, R1
L2:
MUL R3, 1

output=
L2:
```

Q5. Constant Folding, Constant Propagation and Dead Code Elimination

Idea: Walk forward, replacing known constants and evaluating expressions whose operands are both constants. Then walk backward and remove assignments whose result is never used (not live).

Problem Statement Write a C program that optimizes a basic block.

The program should:

1. Read `n`, then `n` statements: `x = y op z` or `x = y` (operands are integer constants or variables, op is `+ - *`).
2. Read the live-out variables on the last line as `m v1 ... vm`.
3. **Forward pass:** replace variables with known constant values, and fold an expression when both operands are constants. A variable assigned a non-constant value is no longer constant.
4. **Backward pass:** remove statements whose left-hand side is not live afterwards.

A kept statement makes its operand variables live.

5. Print the remaining statements in original order.

Input Constraints

- $1 \leqslant n \leqslant 30$

Public Test 1

```
input=
4
a = 3
b = a + 2
c = b * x
d = a + b
2 c d

output=
c = 5 * x
d = 8
```

Public Test 2

```
input=
5
x = 2
y = x * 3
z = y + w
x = z - y
t = y * y
1 x

output=
z = 6 + w
x = z - 6
```

PART C: LOOPS IN FLOW GRAPHS AND LOOP OPTIMIZATION

Q6. Dominators, Back Edges and Natural Loops

Idea: Node `d` dominates `n` if every path from the entry to `n` passes through `d`. An edge `t→h` is a back edge if `h` dominates `t`. The natural loop of a back edge is `h` plus every node that can reach `t` without passing through `h`.

Problem Statement Write a C program that, for a given flow graph, finds the dominators of each node, the back edges, and the natural loop of each back edge.

The program should:

1. Read `n` (nodes `1..n`, node 1 is the entry) and `m` (edges), followed by `m` edges `u v`.
2. Compute dominators with the iterative method: `Dom(1) = {1}`, `Dom(k) = {k} ∪ ∩ Dom(p)` over all predecessors `p`.
3. Print `Dominators:` and one line per node as `k: d1 d2 ...`.
4. Print back edges in input order (`none` if there are none).
5. Print the natural loop (nodes in ascending order) for each back edge.

Input Constraints

- $1 \leqslant n \leqslant 20$; every node is reachable from node 1.

Public Test 1

```
input=
6 7
1 2
2 3
3 4
4 3
4 5
5 2
5 6

output=
Dominators:
1: 1
2: 1 2
3: 1 2 3
4: 1 2 3 4
5: 1 2 3 4 5
6: 1 2 3 4 5 6
Back edges: 4->3 5->2
Loop (4->3): 3 4
Loop (5->2): 2 3 4 5
```

Public Test 2

```
input=
3 3
1 2
2 2
2 3

output=
Dominators:
1: 1
2: 1 2
3: 1 2 3
Back edges: 2->2
Loop (2->2): 2
```

Q7. Loop-Invariant Code Motion

Idea: A statement whose value does not change across iterations can be moved out of the loop into a preheader.

Problem Statement Write a C program that finds loop-invariant statements in a loop body and moves them to the preheader.

A statement `x = y op z` is loop-invariant if:

- `x` is defined **only once** inside the loop, and
- each operand is a constant, or a variable not defined in the loop, or a variable defined by an already-invariant statement.

(Assume all other safety conditions, such as dominance of exits, are satisfied.)

The program should:

1. Read `n` and then `n` statements of the loop body (`x = y op z`).
2. Repeat the invariance check until no new statement becomes invariant.
3. Print `Preheader:` followed by the invariant statements in original order (`(none)` if there are none).
4. Print `Loop body:` followed by the remaining statements in original order.

Public Test 1

```
input=
5
t1 = a + b
t2 = t1 * 4
i = i + 1
t3 = t2 + i
s = s + t3

output=
Preheader:
t1 = a + b
t2 = t1 * 4
Loop body:
i = i + 1
t3 = t2 + i
s = s + t3
```

Public Test 2

```
input=
3
x = 5
y = x + 1
x = y

output=
Preheader:
(none)
Loop body:
x = 5
y = x + 1
x = y
```

Q8. Strength Reduction on Induction Variables

Idea: In a loop where `i` increases by a constant each iteration, the product `t = i * k` also changes by a constant each iteration, so the multiplication can be replaced by an addition.

Problem Statement Write a C program that performs strength reduction for a loop with one basic induction variable.

The program should:

1. Read two statements: `i = i + c` and `t = i * k`, in that order (`c` and `k` are positive integers).

2. Print `Preheader:` with the initialization `t = i * k`.
3. Print `Loop body:` with `i = i + c` and the replacement `t = t + (c*k)` (the product printed as a number).

Public Test 1

```
input=
2
i = i + 1
t = i * 4

output=
Preheader:
t = i * 4
Loop body:
i = i + 1
t = t + 4
```

Public Test 2

```
input=
2
i = i + 2
t = i * 3

output=
Preheader:
t = i * 3
Loop body:
i = i + 2
t = t + 6
```

PART D: DATA FLOW ANALYSIS

All three questions use the iterative algorithm: start with the initial values, repeat the equations for every block until no set changes.

Q9. Reaching Definitions

Equations: `IN[B] = ∪ OUT[P]` over predecessors `P`; `OUT[B] = GEN[B] ∪ (IN[B] - KILL[B])`. Initialize `OUT[B] = {}` for all blocks.

Problem Statement Write a C program that computes `IN` and `OUT` of reaching definitions for every block.

The program should:

1. Read `n m`, then `m` edges `u v`.

2. Then read `n` lines (block 1 to n), each as `g gen_1..gen_g k kill_1..kill_k` (definition numbers).
3. Iterate until no change and print `Bi: IN={...} OUT={...}`.

Public Test 1

Block	GEN	KILL
B1	{1,2}	{3,4,5}
B2	{3}	{1,5}
B3	{4}	{2}
B4	{5}	{1,3}

```
input=
4 4
1 2
2 3
2 4
3 2
2 1 2 3 3 4 5
1 3 2 1 5
1 4 1 2
1 5 2 1 3

output=
B1: IN={} OUT={1,2}
B2: IN={1,2,3,4} OUT={2,3,4}
B3: IN={2,3,4} OUT={3,4}
B4: IN={2,3,4} OUT={2,4,5}
```

Public Test 2

```
input=
2 1
1 2
1 1 1 2
1 2 1 1

output=
B1: IN={} OUT={1}
B2: IN={1} OUT={2}
```

Q10. Live Variable Analysis

Equations (backward): `OUT[B] = ∪ IN[S]` over successors `S`; `IN[B] = USE[B] ∪ (OUT[B] − DEF[B])`. Initialize `IN[B] = {}`; `OUT` of the exit block is `{}`.

Problem Statement Write a C program that computes live variables `IN` and `OUT` for every block.

The program should:

1. Read `n m`, then `m` edges `u v`.
2. Then read `n` lines, each as `u u_1..u_u d d_1..d_d` (USE count and variables, then DEF count and variables).
3. Print `Bi: IN={...} OUT={...}` with variables in alphabetical order.

Public Test 1

Block	USE	DEF
B1	{}	{a,b}
B2	{a}	{c}
B3	{b,c}	{a}
B4	{a,c}	{}

```
input=
4 4
1 2
2 3
2 4
3 2
0 2 a b
1 a 1 c
2 b c 1 a
2 a c 0

output=
B1: IN={} OUT={a,b}
B2: IN={a,b} OUT={a,b,c}
B3: IN={b,c} OUT={a,b}
B4: IN={a,c} OUT={}
```

Public Test 2

```
input=
2 1
1 2
0 1 x
1 x 1 y

output=
B1: IN={} OUT={x}
B2: IN={x} OUT={}
```

Q11. Available Expressions

Equations (forward, intersection): `IN[B1] = {}`; `IN[B] = ∩ OUT[P]` over predecessors; `OUT[B] = GEN[B] ∪ (IN[B] − KILL[B])`. Initialize `OUT[B] = U` (all expressions) for every block except the entry.

Problem Statement Write a C program that computes `IN` and `OUT` of available expressions for every block.

The program should:

1. Read `n m u` (blocks, edges, number of expressions numbered `1..u`), then `m` edges `a b`.
2. Then read `n` lines, each as `g gen_1..gen_g k kill_1..kill_k`.
3. Print `Bi: IN={...} OUT={...}`.

Public Test 1

Block	GEN	KILL
B1	{1,2}	{}
B2	{3}	{1}
B3	{4}	{2}
B4	{}	{}

```
input=
4 4 4
1 2
1 3
2 4
3 4
2 1 2 0
1 3 1 1
1 4 1 2
0 0

output=
B1: IN={} OUT={1,2}
B2: IN={1,2} OUT={2,3}
B3: IN={1,2} OUT={1,4}
B4: IN={} OUT={}
```

Public Test 2 (with a loop)

```
input=
3 3 2
1 2
2 3
3 2
1 1 0
1 2 1 1
0 1 2

output=
B1: IN={} OUT={1}
B2: IN={} OUT={2}
B3: IN={2} OUT={}
```

PART E: CODE GENERATION AND SECURITY CHECKING FOR A VIRTUAL MACHINE

Q12. Naive Code Generator for a Stack-Based Virtual Machine

Idea: Translate each three-address statement independently: push operands, apply the operator, pop the result into the destination.

Problem Statement Write a C program that generates stack-machine code from three-address code.

Virtual machine instructions: `PUSH v`, `POP x`, `ADD`, `SUB`, `MUL`, `DIV`.

The program should:

1. Read `n`, then `n` statements: `x = y op z` or `x = y` (operands are variables or integer constants).
2. For `x = y op z` print: `PUSH y`, `PUSH z`, the operator instruction, `POP x`.
3. For `x = y` print: `PUSH y`, `POP x`.

Public Test 1

```
input=
3
t1 = a + b
t2 = t1 * 4
x = t2

output=
PUSH a
PUSH b
ADD
POP t1
PUSH t1
PUSH 4
MUL
POP t2
PUSH t2
POP x
```

Public Test 2

```
input=
2
t1 = x - 3
y = t1 / 2

output=
PUSH x
PUSH 3
SUB
POP t1
PUSH t1
PUSH 2
DIV
POP y
```

Q13. Naive Code Generator for a Register-Based Virtual Machine

Problem Statement Write a C program that generates register-machine code from three-address code, using only registers `R0` and `R1`.

Instructions: `LD R,x` (load variable), `LDI R,c` (load integer constant), `ST x,R`, and `ADD`, `SUB`, `MUL`, `DIV` in the form `OP Rd, Rs1, Rs2`.

The program should:

1. Read `n`, then `n` statements: `x = y op z` or `x = y`.
2. For `x = y op z`: load `y` into `R0`, load `z` into `R1`, perform `OP R0, R0, R1`, then `ST x, R0`.
3. For `x = y`: load `y` into `R0`, then `ST x, R0`.
4. Use `LD` for variables and `LDI` for integer constants.

Public Test 1

```
input=
3
t1 = a + b
t2 = t1 * 4
x = t2

output=
LD R0, a
LD R1, b
ADD R0, R0, R1
ST t1, R0
LD R0, t1
LDI R1, 4
MUL R0, R0, R1
ST t2, R0
LD R0, t2
ST x, R0
```

Public Test 2

```
input=
2
t1 = x - 3
y = t1 / 2

output=
LD R0, x
LDI R1, 3
SUB R0, R0, R1
ST t1, R0
LD R0, t1
LDI R1, 2
DIV R0, R0, R1
ST y, R0
```

Q14. Security Checker (Bytecode Verifier) for Virtual Machine Code

Idea: Before running untrusted VM code, simulate it statically and track the stack depth at every instruction. Reject any program that could underflow or overflow the stack, jump outside the program, reach an instruction with two different stack depths, or run off the end.

Problem Statement Write a C program that verifies a stack-machine program.

Instructions (numbered from 0) and their stack effect:

Instruction	Needs	Effect
`PUSH c`	none	depth + 1
`LOAD v`	none	depth + 1
`STORE v`	depth ⩾ 1	depth − 1
`ADD`, `SUB`, `MUL`	depth ⩾ 2	depth − 1
`JMP t`	none	continues at `t`
`JZ t`	depth ⩾ 1	depth − 1; continues at `t` or at the next instruction
`HALT`	none	stops

The program should:

1. Read `MAX` (maximum allowed stack depth), then `n`, then `n` instructions.
2. First check that every jump target is in `0..n-1`. Otherwise print `ERROR: invalid jump target at instruction k`.

3. Simulate from instruction 0 following every path (fall-through first, then jump target), recording the stack depth at each instruction.
4. Report the first error found in execution order:
   - `ERROR: stack underflow at instruction k`
   - `ERROR: stack overflow at instruction k` (depth would exceed `MAX`)
   - `ERROR: inconsistent stack depth at instruction k` (reached with two different depths)
   - `ERROR: control reaches end of program without HALT`
5. If no error is found, print `SAFE`.

Input Constraints

- $1 \leqslant n \leqslant 50, 1 \leqslant MAX \leqslant 20$

Public Test 1

```
input=
2
5
PUSH 5
PUSH 7
ADD
STORE x
HALT

output=
SAFE
```

Public Test 2 (underflow)

```
input=
2
4
PUSH 1
ADD
STORE x
HALT

output=
ERROR: stack underflow at instruction 1
```

Public Test 3 (overflow)

```
input=
2
4
PUSH 1
PUSH 2
PUSH 3
HALT

output=
ERROR: stack overflow at instruction 2
```

Public Test 4 (invalid jump)

```
input=
2
2
JMP 5
HALT

output=
ERROR: invalid jump target at instruction 0
```

Public Test 5 (missing HALT)

```
input=
2
2
PUSH 1
STORE x

output=
ERROR: control reaches end of program without HALT
```

Public Test 6 (inconsistent stack depth at a merge point)

```
input=
3
5
PUSH 1
JZ 4
PUSH 5
JMP 4
HALT

output=
ERROR: inconsistent stack depth at instruction 4
```